



موضوع

پروژه برنامه‌نویسی

اعضای گروه

فریده مطیعان ۴۰۳۲۰۵۴۶۳

نام درس

برنامه‌ریزی تصادفی

نام استاد

آقای دکتر رفیعی

بهار ۱۴۰۵

فهرست

۳	 مرور کلی پروژه
۳	 اهداف اصلی پروژه
۳	 روش L-Shape Multicut
۵	 معرفی مساله کشاورز
۵	 مدلسازی مساله
۶	 توضیح کدها
۶	 کتابخانه‌ها
۶	 پارامترهای مساله
۷	 پارامترهای دارای عدم قطعیت
۷	 تولید داده های تصادفی
۸	 کد بخش تولید اعداد تصادفی
۱۰	 خروجی کد
۱۱	 محاسبه ماتریس‌های ضرایب متغیر مرحله اول (t) و سمت راست در زیرمسائل (h)
۱۲	 منطق کلی کد الگوریتم
۱۳	 تعریف متغیرهای گلوبال
۱۳	 راه‌اندازی مستر پرابلم
۱۴	 حلقه ویل
۱۵	 فیزیبیلیتی چک
۱۸	 ایجاد برش بهینگی
۲۰	 بررسی بهینگی کلیه سناریوها
۲۰	 نتیجه اجرا بر داده های ۵۰ تایی
۲۳	 نمودار

مرور کلی پروژه

در این پروژه، روش L-Shape Multicut برای مسئله کشاورز بررسی می‌شود. ابتدا مدل‌سازی مسئله و نحوه نوشتن مسئله مستر و زیر مسئله و دوگان زیر مسئله بررسی می‌شود. در بخش دوم روش L-Shape Multicut به صورت اجمالی معرفی می‌شوند. در بخش سوم کد نویسی بررسی می‌شود.

اهداف اصلی پروژه

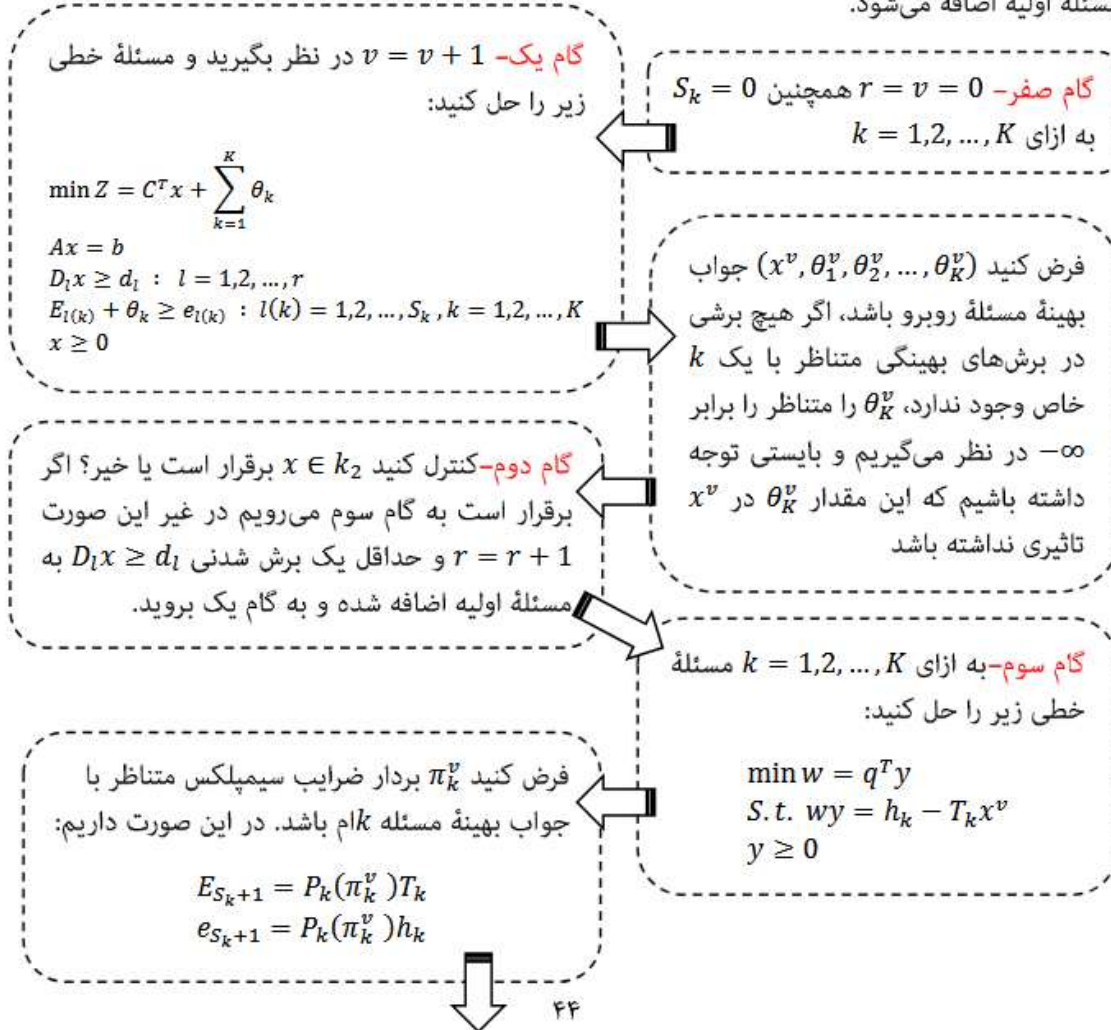
با توجه به اهمیت حل سوال تصادفی در زمان مناسب زمانی که ابعاد مسئله بالا است، روش‌های متفاوتی توسعه یافته‌اند یکی از این روش‌ها روش L-Shape Multicut است. در این پروژه قرار است ابتدا ۵۰ سناریو ساخته شده و با این روش حل شوند

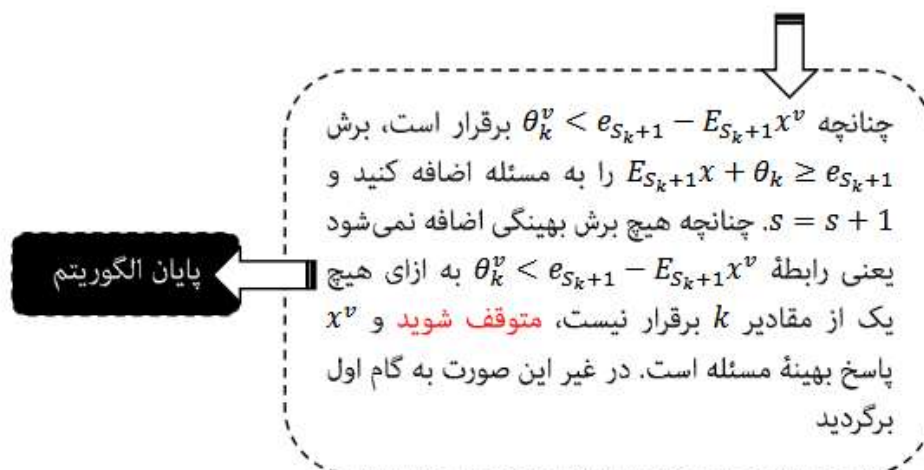
روش L-Shape Multicut.

فرآیند کلی روش به صورت چارت زیر است. این صفحه از جزوه دکتر رفیعی آورده شده است.

الگوریتم I شکل چند برشی

در الگوریتم قبلی (تک برشی)، در هر حل بهینه از تمامی k تحقق از مسئله مرحله دوم به دست می‌آمد و ضرایب سیمپلکس متناظر با جواب‌های بهینه در هر دور منجر به اضافه شدن تنها یک برش بهینگی به مسئله می‌شد. در این الگوریتم چندین برش بهینگی را به جای یک برش در هر تکرار در نظر می‌گیریم، در واقع در هر دور به ازای هر تحقق بردار تصادفی یک برش در گام سوم اضافه می‌شود یعنی در هر تکرار، حداکثر k برش به مسئله اولیه اضافه می‌شود.





معرفی مساله کشاورز

یک کشاورز باید سر سال تصمیم بگیرد چه حجمی از زمین خود را به سه محصول گندم ذرت و چغندر تخصیص دهد. در زمان برداشت او باید از محصولات گندم و ذرت حداقل مقداری داشته باشد و بقیه محصولات را می تواند بفروشد یا بخرد. همچنین تا زیر مقدار مشخصی چغندر قیمت مطلوب تری دارد و مقادیر بالای آن به قیمت پایین تری خرید می شود.

برای مطالعه بیشتر به فصل اول کتاب مرجع مراجعه کنید.

مدلسازی مساله

برای مدلسازی نیاز به مسئله مستر پرابلم داریم که به صورت زیر نوشته می شود.

$$\min 150x_1 + 230x_2 + 260x_3 + \sum_s \theta_s$$

$$150x_1 + 230x_2 + 260x_3 \leq 500$$

$$E_{nooc+1} x + \theta_s \geq e_{nofc+1}$$

$$D_{nofc+1} x^v > d_{nofc+1}$$

$$x_1 \cdot x_2 \cdot x_3 \geq 0 \cdot \theta_s \text{ url}$$

که برش های بهینگی و موجهی به صورت توضیح داده شده، تولید می شوند.

زیر مسئله برای هر سناریو به صورت زیر نوشته می‌شود. مثلاً اگر در سناریوی پارامترهای زیر حادث شوند. مدل‌سازی به صورت زیر است.

Crop_harvest_coefficients = [3, 3.6, 24]

least_need = [200, 240]

selling_price = [-170, -150, -36, -10]

purchase_price = [238, 210]

$$\begin{aligned} \min & 170w_1 - 238y_1 + 150w_2 - 210y_2 + 36w_3 + 10w_4 \\ & + y_1 - w_1 \geq 200 - 3x_1 \\ & + y_2 - w_2 \geq 240 - 3.6x_2 \\ & -w_3 - w_4 \geq -24x_3 \\ & -w_3 \geq -6000 \end{aligned}$$

توضیح کدها

کتابخانه‌ها

سه کتابخانه `pyomo` برای بهینه‌سازی `matplotlib` نمایش نمودارها و `numpy` اعمال جبری استفاده شده است. و کد بر لب تاب با کور ای سون و رم ۱۶ اجرا شده است.

```
from pyomo.environ import *
import numpy as np
import matplotlib.pyplot as plt
```

پارامترهای مساله

سه پارامتر کل مساحت زمین، حداکثر مقدار چغندر با نرخ مطلوب به کیلوگرم، و هزینه کشت در سال اول برای محصولات گندم، ذرت و چغندر معرفی شده‌اند. این مقادیر عدم قطعیت ندارند و در مرحله اول کاربرد دارند.

```
total_area_for_planting = 500
farmer_quota = 6000
planting_cost_per_acre = [ 150, 230, 260]
```

پارامترهای دارای عدم قطعیت

حداقل مقدار مورد نیاز ذرت و گندم، قیمت خرید و فروش محصولات و بهره وری محصولات دارای عدم قطعیت هستند. در کد دو داده تست تعریف شده اند. داده `tast_data_book_farmer_problem` که سه سناریو کتاب را شامل است و داده `tast_data_50` که ۶۴ سناریو مختلف می سازد (با گرفتن تعداد بازه هر متغیر تصادفی به توان دو این عدد سناریو می سازد).

نحوه تولید `tast_data_50` در زیر شرح داده شده است. ابتدا توزیع ریاضی و سپس کد شرح داده می شود.

تولید داده های تصادفی

برای تولید سناریوها و با توجه به وابستگی متغیرهای بهره وری مزرعه و هزینه خرید و فروش روابط زیر در نظر گرفته می شود.

$$\text{گندم بهره وری} = 2.5 + 2.5\delta$$

$$\text{ذرت بهره وری} = 3 + 3\delta$$

$$\text{چقندر بهره وری} = 20 + 20\delta$$

برای سادگی فرض می شود بهره وری برای سه محصول از توابع بالا پیروی می کند. اما برای قیمت فروش و خرید برای بالا بردن دقت فرض می کنیم تابع بهره وری و ترمی از خطا است. (عواملی به جز بهره وری) توجه کنید که امگا و سایر ترم های خطا از توزیع نرمال با میانگین صفر و واریانس ۰,۳۳ پیروی می کنند.

$$(\text{ذرت}) = 235(210) - 23.5(21)\delta + 23.5(21)\theta(\vartheta)$$

$$\begin{aligned} & \left[\text{قیمت فروش} \left[\text{گندم، ذرت، غیر جذاب، چقندر جذاب} \right] \right. \\ & = [-170, -150, -36, -10] - 0.1[-170, -150, -36, -10]\delta \\ & \quad \left. + [-170, -150, -36, -10][\alpha, \beta, \gamma, \varepsilon] \right] \end{aligned}$$

برای تولید داده از روش قاعده دوزنقه نعیم یافته استفاده شده است. به این شرح که مثلاً برای δ که از توزیع نرمال پیروی می کند ۱۰۰۰ نمونه گرفته می شود. و توزیع را به تعدادی بازه یکسان (تعداد سناریوای که می خواهیم) می شکنند. تعداد نقاطی که در هر بازه قرار گرفته را می شمارند و میانگین نقاط هر داده را محاسبه

می‌کنند. این تعداد را بر کل نمونه گیری‌ها تقسیم می‌شود و احتمال به دست آورده شده را به میانگین داده های آن بازه نسبت می‌دهند. مثلاً به احتمال ده درصد δ برابر با ۵ می‌شود.

با ۷ متغیر تصادفی و بازه بندی ۵ تایی برای هر متغیر ۵ به توان ۷ سناریو ساخته می‌شود که زیاد است و چون بار اصلی پروژه این بخش نیست، ترم خطا برای همه خرید و فروش‌ها برابر هم و با متغیر α مدل می‌شود پس تنها دو متغیر تصادفی δ و α خواهیم داشت. برای هر کدام ۸ بازه و مجموعاً ۶۴ سناریو ساخته شده است.

برای متغیرهای سمت راست یا حداقل مورد نیاز ذرت و گندم برای تمام حالات مقدار قطعی ۲۰۰ و ۲۴۰ منظور گشته است. اما مدلسازی به گونه ای است که می‌توان مقادیر مختلف را به آن وارد کرد.

علت این امر این است که از افزایش نمایی سناریوها جلوگیری کنیم.

قابل بیان است شیوه های متفاوتی برای تولید سناریو ها از جمله نمونه گیری شرطی ، مونت کارلو، انطباق گشتاورها و روش های متفاوتی برای کاهش سناریوها از جمله گشتاورها، کی مینز و کاهش سناریو پسرو پیشرو وجود دارد اما چون تمرکز پروژه بر این موارد نیست از انجام آن خودداری شده است.

کد بخش تولید اعداد تصادفی

بخش اول پارامترها و متغیرهای تابع را تعریف می‌کند.

```
def fast_data_50(num_bins = 8):
    global least_need, crop_harvest_coefficients, purchase_price, selling_price, scenario, scenario_prob
    scenario_prob = {}
    crop_harvest_coefficients = {}
    least_need = {}
    selling_price = {}
    purchase_price = {}

    crop_harvest_coefficients_initial = [2.5, 3.0, 20.0]
    selling_price_initial = [-170, -150, -36, -10]
    purchase_price_initial = [238, 210]

    num_samples = 20000
    # num_bins = 8
    lower_bound = -1.0
    upper_bound = 1.0
    mu = 0
    sigma = 0.33
    scenario = num_bins * num_bins
```

این تکه کد از تابع نرمال هر کدام ۲۰۰۰ نمونه می‌گیرد. و چون ما بازهای حداقل و حداکثر ۱ و -۱ تعریف کردیم اگر نمونه ای خارج از این بازه شود تابع وایل آن را خارج و نمونه گیری مجدد انجام می‌دهد و این کار تا جایی انجام می‌دهد که تعداد سمپل به ۲۰۰۰ برسد

```

# making omega and error term
samples_x = np.random.normal(mu, sigma, num_samples)
samples_y = np.random.normal(mu, sigma, num_samples)

mask = (samples_x >= lower_bound) & (samples_x <= upper_bound) & \
       (samples_y >= lower_bound) & (samples_y <= upper_bound)
samples_x = samples_x[mask]
samples_y = samples_y[mask]

while len(samples_x) < num_samples:
    new_x = np.random.normal(mu, sigma, num_samples - len(samples_x))
    new_y = np.random.normal(mu, sigma, num_samples - len(samples_x))
    mask = (new_x >= lower_bound) & (new_x <= upper_bound) & \
           (new_y >= lower_bound) & (new_y <= upper_bound)
    samples_x = np.concatenate([samples_x, new_x[mask]])
    samples_y = np.concatenate([samples_y, new_y[mask]])

samples_x = samples_x[:num_samples]
samples_y = samples_y[:num_samples]

```

سپس داده ها با توجه به مقدار امگا و الفا(ایکس و وای کد) در یکی از بازه ها قرار می گیرند

```

bin_edges = np.linspace(lower_bound, upper_bound, num_bins + 1)

scenarios = []
total_samples = len(samples_x)

for i in range(num_bins):
    for j in range(num_bins):
        if i == num_bins - 1 and j == num_bins - 1:
            mask = (samples_x >= bin_edges[i]) & (samples_x <= bin_edges[i+1]) & \
                   (samples_y >= bin_edges[j]) & (samples_y <= bin_edges[j+1])
        elif i == num_bins - 1:
            mask = (samples_x >= bin_edges[i]) & (samples_x <= bin_edges[i+1]) & \
                   (samples_y >= bin_edges[j]) & (samples_y < bin_edges[j+1])
        elif j == num_bins - 1:
            mask = (samples_x >= bin_edges[i]) & (samples_x < bin_edges[i+1]) & \
                   (samples_y >= bin_edges[j]) & (samples_y <= bin_edges[j+1])
        else:
            mask = (samples_x >= bin_edges[i]) & (samples_x < bin_edges[i+1]) & \
                   (samples_y >= bin_edges[j]) & (samples_y < bin_edges[j+1])

        points_x = samples_x[mask]
        points_y = samples_y[mask]
        count = len(points_x)

```

در ادامه اگر در بازه ای نقطه ای نباشد میانگین ایکس و وای با احتمال ۰ ثبت می شود و اگر در بازه نقطه ای باشند تعداد نقاط بر ۴۰۰۰ تقسیم می شوند که احتمال اعداد این بازه را نشان می دهد. مقدار میانگین تتا و الفا این نقاط، تتا و الفا این سناریو را نمایش می دهند. سپس مقدار میانگین تتا و میانگین الفا و احتمالشان ذخیره می شود.

```
count = len(points_x)

if count == 0:
    mean_x = (bin_edges[i] + bin_edges[i+1]) / 2
    mean_y = (bin_edges[j] + bin_edges[j+1]) / 2
    probability = 0
else:
    mean_x = np.mean(points_x)
    mean_y = np.mean(points_y)
    probability = count / total_samples

scenarios.append({
    'x_bin': (bin_edges[i], bin_edges[i+1]),
    'y_bin': (bin_edges[j], bin_edges[j+1]),
    'count': count,
    'probability': probability,
    'mean_x': mean_x,
    'mean_y': mean_y
})

print(f"تعداد کل نمونه های معتبر: {total_samples}")
print("=" * 100)
print(f"{'X بازه':<16} {'Y بازه':<16} {'تعداد':<8} {'احتمال':<12} {'X میانگین':<12} {'Y میانگین':<12}")
print("=" * 100)

total_prob = 0
```

کدهای بعد نیز مقادیر را در قالب جدول نمایش می دهند.

یک لیست به اسم امگا_ارور برای هر سناریو این مقادیر را در یک لیست می ریزد. از این مقادیر با توجه به فرمول های بیان شده در بخش قبل مقادیر پارامتری هر سناریو معلوم می شود.

```
omega_error = zip([f'{x:.4f}' for x in x_values], [f'{y:.4f}' for y in y_values], [f'{p:.4f}' for p in p_values])
omega_error = [i for i in omega_error]
print(f"{'':<100}")
print(omega_error)
for enu, s in enumerate(omega_error):
    scenario_prob[enu+1] = float(omega_error[enu][2])
    white_harvest_coefficients = Crop_harvest_coefficients_initial[0] + Crop_harvest_coefficients_initial[0]*float(omega_error[enu][0])
    crop_harvest_coefficients = Crop_harvest_coefficients_initial[1] + Crop_harvest_coefficients_initial[1]*float(omega_error[enu][0])
    bite_harvest_coefficients = Crop_harvest_coefficients_initial[2] + Crop_harvest_coefficients_initial[2]*float(omega_error[enu][0])
    selling_price_initial = [i*0.1*float(omega_error[enu][0]) + 0.1*float(omega_error[enu][1]) for i in selling_price_initial]
    purchase_price_initial = [i*0.1*float(omega_error[enu][0]) + 0.1*float(omega_error[enu][1]) for i in purchase_price_initial]
    crop_harvest_coefficients[enu+1] = (white_harvest_coefficients, crop_harvest_coefficients, bite_harvest_coefficients)
    selling_price[enu+1] = selling_price_initial
    purchase_price[enu+1] = purchase_price_initial
    least_need[enu+1] = [200, 240]
```

خروجی کد

least_need, Crop_harvest_coefficients, purchase_price, selling_price, scenario,

scenario_prob هر کدام به ترتیب حداقل مورد نیاز، بهره وری محصولات گندم و ذرت و چغندرریال قیمت خرید گندم و ذرت و هزینه فروش گندم، ذرت، چغندر مطلوب و غیرمطلوب و احتمال هر سناریو را در قالب دیکشنری نگه می‌دارد.

کلید دیکشنری شماره سناریو و ولیو دیکشنری مقادیر بالا است. نمونه خروجی:

least_need

```
{1: [200, 240],
 2: [200, 240],
 3: [200, 240],
 4: [200, 240],
 5: [200, 240],
 6: [200, 240],
 7: [200, 240],
 8: [200, 240],
 9: [200, 240],
10: [200, 240],
11: [200, 240],
12: [200, 240],
13: [200, 240],
14: [200, 240],
15: [200, 240],
16: [200, 240],
17: [200, 240],
```

محاسبه ماتریس‌های ضرایب متغیر مرحله اول (t) و سمت راست در زیرمسائل (h) در اولین گام توجه کنید مدلسازی به قالب کانونی نوشته شده است به گونه ای که :

$$wy(\xi) \geq h(\xi) - t(\xi)x$$

اما حتی اگر به قالب کانونی هم نوشته نمی‌شد به مشکلی بر نمی‌خوریم. زیرا متغیر دوگان علامت عکس می‌گیرد.

. از جایی که ساختار زیرمسائل در تکرارهای الگوریتم یکسان است میتوان از تابع زیر استفاده کرد و قبل حل بندرز از آن استفاده کرد. لزومی ندارد حتما مدل را بسازیم و ضرایب را مستقیما استخراج کنیم.

این کار سبب می شود نسبت به کد تدریس شده لازم نباشد در هر تکرار سناریو ماتریس محاسبه شوند.

```
h = {}
for s in range(1,scenario+1):
    h[s] = [least_need[s][0],least_need[s][1],0,-1*farmer_quota]
h

t = {}
for s in range(1,scenario+1):
    scenario_t = np.array([[Crop_harvest_coefficients[s][0],0,0],
                           [0,Crop_harvest_coefficients[s][1],0],
                           [0,0,Crop_harvest_coefficients[s][2]],
                           [0,0,0]])
    t[s] = scenario_t
```

که خروجی مشابه زیر برای سه سناریو می دهد.

```
t:{1: array([[ 2.5,  0.,  0. ],
            [ 0.,  3.,  0. ],
            [ 0.,  0., 20. ]]), 2: array([[ 3.,  0.,  0. ],
            [ 0.,  3.6,  0. ],
            [ 0.,  0., 24. ]]), 3: array([[ 2.,  0.,  0. ],
            [ 0.,  2.4,  0. ],
            [ 0.,  0., 16. ]]), h:{1: [200, 240, 0, -6000], 2: [200, 240, 0, -6000], 3: [200, 240, 0, -6000]}}
```

منطق کلی کد الگوریتم

به دلیل بزرگی کد ابتدا منطق بخش های مختلف کد به صورت کلی را می آوریم و هر بخش را به صورت کامل با کد شرح می دهیم. کد اصلی بخش های زیر را دارد.

تعریف متغیرهای گلوبال

تعریف و پیاده سازی اولیه تابع مستر

ورود به حلقه وایل

-حل تابع مستر

-چک کردن فیزیبل بودن برای هر سناریو:

--بررسی موجه بودن جواب مستر پرابلم برای زیر مسئله
 ----اضافه کردن کات موجهی در صورت موجه نبودن و خروج از حلقه
 -ایجاد برش های بهینگی برای هر سناریو:
 --حل کردن زیر مسئله
 --محاسبه دلیو
 --بررسی تکرار نشدن دلیو:
 ----ایجاد برش بهینگی
 -ایا هیچ برشی از تمام سناریو ها اضافه نشد؟
 --به نقطه بهینه رسیدیم . حلقه وایل متوقف شود .

تعریف متغیرهای گلوبال

متغیرها تعداد تکرار ها با وی تعداد برش های فیزیبیلیتی nofc و تعداد برش های بهینگی nooc با متغیرهای تعریف می شود.

دیکشنری تتا شامل کلید و ولیو است. کلید شماره سناریو را نمایش می دهد و ولیو شامل یک لیست هست که مقدار تتا مستر پرابلم این سناریو را در ایتريشن های مختلف ذخیره می کند.

لیست دلیو هم مشابه تتا است و مقادیر دلیو را نگه می دارد.

$$W = e-Ex$$

```
# whole algorithm
nofc, nooc, v = (0, 0, 0)
theta_values = {}
w = {}

for s in range(1,scenario+1):
    w[s]=[]
```

راه اندازی مستر پرابلم

ابتدا مدل را میسازیم و برای سه محصول ایندکس ای را تعریف می کنیم. و متغیر متراژ هر محصول ساخته می شود. برای هر سناریو نیاز به یک عبارت تتا داریم. پس ایندکس و متغیر تتا را برای هر سناریو می سازیم.

تابع هدف و محدودیت اول تعریف می‌شود. هنوز محدودیتی از کات های بهینگی و موجهی نداریم اما نیاز به آن ها در ادامه هست پس لیستی از محدودیت ها را برای هر کدام می‌سازیم. برای دور اول لازم است تنها مقادیر منفی بسیار بزرگ داشته باشند که لیست آن را از الان ایجاد کرده تا در خطوط بعدی برای تکرار اول مقدار آن را عدد منفی بسیار بزرگ بگذاریم.

$$\min 150x_1 + 230x_2 + 260x_3 + \sum_s \theta_s$$

$$150x_1 + 230x_2 + 260x_3 \leq 500$$

$$x_1 \cdot x_2 \cdot x_3 \geq 0 \cdot \theta_s \text{ url}$$

```
for s in range(1,scenario+1):
    w[s]=[]

#master initialization
master = ConcreteModel()
master.i = RangeSet(3)
master.x = Var(master.i , domain=NonNegativeReals)
master.scenario = RangeSet(scenario)
master.theta = Var(master.scenario , domain=Reals)

master.obj = Objective(expr =sum( planting_cost_per_acre[i-1]*master.x[i] for i in master.i)
                        + sum(master.theta[s] for s in master.scenario))

def rule_c1(master):
    return sum(master.x[i] for i in master.i) <= total_area_for_planting
master.c1 = Constraint( rule= rule_c1)
master.optimality_cut = ConstraintList()
master.feasibility_cut = ConstraintList()
master.theta_initialization = ConstraintList()
```

حلقه ویل

این حلقه الگوریتم را تا زمانی که به جواب بهینه یا ناموجه نرسیم اجرا میکند. یک کانتر تعریف شده است که اگر مسئله همگرا نشد نهایت اجازه اجرا ۵۰ ایتريشن را داشته باشد.

در هر ایتريشن به تعداد تایتريشن یکی اضافه می‌شود. اگر در تکرار اول باشیم، محدودیتی ایجاد می‌شود که به کل تنها مقدار منفی بزرگی بدهد. اما در تکرار های بعد که برش اضافه می‌شود مقادیر تنها به مرور بزرگتر می‌شوند این محدودیت باید حذف شود. سپس مستر پرابلم اجرا می‌شود و ماتریسی از ایکس ها را خروجی می‌دهد.

اگر مستر پرابلم ناموجه شود، برنامه متوقف شده و خطای ناموجه بودن را نشان می‌دهد. در غیر اینصورت مراحل ادامه می‌یابند.

قسمت آخر نیز شماره تکرار و مقدار متغیرهای مستر پرابلم را نمایش می‌دهد.

```
v += 1
if v == 1:
    master.theta_initialization = ConstraintList()
    for enum, s in enumerate(master.scenario):
        master.theta_initialization.add (master.theta[s] == -100000 )
        theta_values[s] = [-100000]
else:
    master.theta_initialization.deactivate()
    for enum, s in enumerate(master.scenario):
        print(f"\n[theta{master.theta[s]}] from previous iteration: {value(master.theta[s]):,.2f}")
        theta_values[s].append(value(master.theta[s]))

solver=SolverFactory('cplex')
result=solver.solve(master, tee=False)
if result.solver.termination_condition == TerminationCondition.infeasible:
    print(f"*****")
    print(f"model is infeasible")
    break

print("\n" + "="*50)
print(f"📦 Iteration {v}")
print("-"*50)
print(f" Master Problem Objective Value : {value(master.obj):,.2f}")
print(f" Decision Variables:")
for i in master.i:
    print(f"{master.x[i]}: {value(master.x[i])}")
print("-"*50)
```

فیزیبیلیتی چک

باید چک شود که آیا این ایکس در تمام سناریو ها موجه است. توجه شود که همه سناریوها باید چک شوند و بعد به سراغ گام بعد که تولید برش بهینگی است رفت. و اگر این دو تکه از هم جدا نباشند مثلاً تا سناریو سه که ایکس موجه است برش ساختیم اما برای سناریو چهار این ایکس موجه نیست آن سه برش باید حذف شوند و هم کد پیچیده تر می‌شود و زمان اجرا افزایش میابد.

برای هر سناریو مدل چک کردن موجه بودن ساخته می‌شود. این مدل همان مدل زیر مسائل سناریوها با مقادیر مثبت و منفی وی است که به شرح زیر آورده می‌شود.

فرض کنید در یک سناریو مقادیر تصادفی رخ می‌دهد .

Crop_harvest_coefficients = [3, 3.6, 24]

least_need = [200 ,240]

selling_price = [-170, -150, -36, -10]

purchase_price = [238, 210]

و مدل مسیله چک کردن بهینگی زیر مسیله این سناریو به شرح زیر است.

$$\min +v^+1 - v^-1 + v^+2 - v^-2 + v^+3 - v^-3 + v^+4 - v^-4$$

$$+ y1 - w1 + v^+1 - v^-1 \geq 200 - 3x1$$

$$+ y2 - w2 + v^+2 - v^-2 \geq 240 - 3.6x2$$

$$-w3 - w4 + v^+3 - v^-3 \geq -24x3$$

$$-w3 + v^+4 - v^-4 \geq -6000$$

البته می توان مستقیما مسئله دوگان زیر مسئله را نوشت و این برش را اضافه کرد.

```
print("-----feasibility check-----")
for s in range(1,scenarios+1):
    print("\n" + "-"*50)
    print(f" scenario[{s}]")
    print("-"*50)

# feasibility check

feasibility_simplex_multiplier = []
dual_feasibility_subproblem = []

feasibility_sub = ConcreteModel()
feasibility_sub.constraint_index = RangeSet(4)
feasibility_sub.sale_index = RangeSet(4)
feasibility_sub.purchase_index = RangeSet(2)

feasibility_sub.positivev = Var(feasibility_sub.constraint_index, domain = NonnegativeReals)
feasibility_sub.negativev = Var(feasibility_sub.constraint_index, domain = NonnegativeReals)
feasibility_sub.w = Var(feasibility_sub.sale_index, domain = NonnegativeReals)
feasibility_sub.y = Var(feasibility_sub.purchase_index, domain = NonnegativeReals)

rule_obj = sum(feasibility_sub.positivev[i] for i in feasibility_sub.constraint_index) + sum(feasibility_sub.negativev[i] for i in feasibility_sub.constraint_index)
feasibility_sub.obj = Objective(expr=rule_obj)

feasibility_sub.c1 = Constraint(expr = feasibility_sub.y[1] + (-1)*feasibility_sub.w[1] + feasibility_sub.positivev[1] - feasibility_sub.negativev [1] >= least_need[s][0] - crop_h
feasibility_sub.c2 = Constraint(expr = feasibility_sub.y[2] + (-1)*feasibility_sub.w[2] + feasibility_sub.positivev[2] - feasibility_sub.negativev [2] >= least_need[s][1] - crop_h
feasibility_sub.c3 = Constraint(expr = -feasibility_sub.w[3] - feasibility_sub.w[4] + feasibility_sub.positivev[3] - feasibility_sub.negativev [3] >= -crop_harvest_coefficients[s][
feasibility_sub.c4 = Constraint(expr = -feasibility_sub.w[3] + feasibility_sub.positivev[4] - feasibility_sub.negativev [4] >= -farmer_quota)
```

در ادامه کد اجرا و مقدار دوگان مسئله استخراج می شود. و مقادیر دوگان چاپ می شوند.

```

feasibility_sub.dual = Suffix(direction=Suffix.IMPORT)
solver = SolverFactory('cplex')
result = solver.solve(feasibility_sub)

#dual feasibility
for c in feasibility_sub.component_objects( Constraint, active = True ):
    cobject = getattr( feasibility_sub, str( c ) )
    for index in cobject:
        dual_feasibility_subproblem.append(feasibility_sub.dual[cobject[index]])

feasibility_simplex_multiplier[s] = dual_feasibility_subproblem

print(f"\n ♦ Scenario {s} feasibility check:")
print(f"      π (Dual values) = {feasibility_simplex_multiplier[s]}")

```

در ادامه مساله اجرا می شود و کد زیر چک می کند حتی اگر یکی از مقادیر وی مثبت یا منفی مقدار گرفته باشد، یک برش موجهی با قاعده زیر ساخته شود.

$$D_{nofc+1} = (\delta_s^v) t_s$$

$$d_{nofc+1} = (\delta_s^v) h_s$$

$$D_{nofc+1} x^v > d_{nofc+1}$$

البته می شود بخش دوگان را در کد خط قبل از ایجاد برش بهینگی زد تا کد بهینه تر شود.

```

# display answer
positive_negative_v_list = []
#print(value(feasibility_sub.obj))
#for i in feasibility_sub.sale_index:
#    print(f"{feasibility_sub.w[i]}: {value(feasibility_sub.w[i])}")
#for i in feasibility_sub.purchase_index:
#    print(f"{feasibility_sub.y[i]}: {value(feasibility_sub.y[i])}")
for i in feasibility_sub.constraint_index:
    positive_negative_v_list.append(value(feasibility_sub.positivev[i]))
    positive_negative_v_list.append(value(feasibility_sub.negativev[i]))
#    print(f"{feasibility_sub.positivev[i]}: {value(feasibility_sub.positivev[i])}")
#    print(f"{feasibility_sub.negativev[i]}: {value(feasibility_sub.negativev[i])}")
positive_negative_v_list
if any(x != 0 for x in positive_negative_v_list):
    print(f"current x is not feasible in scenario number{s}")
    D = np.array([feasibility_simplex_multiplier[s]] @ np.array(t[s]))
    d = np.array([feasibility_simplex_multiplier[s]] @ np.array(h[s]))
    nofc+=1
    master.feasibility_cut.add(list(D @ np.array([master.x[1], master.x[2], master.x[3]]))[0] >= d[0])
    print(f"\n✅ Added optimality cut to master problem.")
    master.feasibility_cut.pprint()
    break
else:
    print(f"current x is feasible.")

```

ایجاد برش بهینگی

پس از اینکه ایکسی تولید شود که در تمام سناریو ها بهینه است وارد این حلقه برای هر سناریو می‌شویم. زیر مسئله حل می‌شود. مثلاً برای داده های زیر زیر مسئله پایین ایجاد شده و حل می‌شود.

Crop_harvest_coefficients = [3, 3.6, 24]

least_need = [200, 240]

selling_price = [-170, -150, -36, -10]

purchase_price = [238, 210]

و مدل مسیله چک کردن بهینگی زیر مسیله این سناریو به شرح زیر است.

$$\min 170w_1 - 238y_1 + 150w_2 - 210y_2 + 36w_3 + 10w_4$$

$$+ y_1 - w_1 + v^+ - v^- \geq 200 - 3x_1$$

$$+ y_2 - w_2 \geq 240 - 3.6x_2$$

$$-w_3 - w_4 \geq -24x_3$$

$$-w_3 \geq -6000$$

همه متغیرها مثبت و پیوسته

```
print(f"-----sub problem solving for feasible x = {value(master.x[1]),value(master.x[2]),value(master.x[3])}-----")
optimal_scenario_counter = 0
for s in range(1,scenario+1):
    print("\n" + "-"*50)
    print(f" scenario{s}")
    print("-"*50)
    # sub problem

    simplex_multiplier = []

    sub = ConcreteModel()
    sub.sale_index = RangeSet(4)
    sub.purchase_index = RangeSet(2)

    sub.w = Var(sub.sale_index, domain = NonnegativeReals)
    sub.y = Var(sub.purchase_index, domain = NonnegativeReals)

    rule_obj = sum(selling_price[s][i-1]*sub.w[i] for i in sub.sale_index) + sum(purchase_price[s][j-1] * sub.y[j] for j in sub.purchase_index)
    sub.obj = (Objective(expr=rule_obj))

    sub.c1 = Constraint(expr = sub.y[1]+ (-1)*sub.w[1]>= least_need[s][0] - Crop_harvest_coefficients[s][0]* value(master.x[1]))
    sub.c2 = Constraint(expr = sub.y[2]+ (-1)*sub.w[2]>= least_need[s][1] - Crop_harvest_coefficients[s][1]* value(master.x[2]))
    sub.c3 = Constraint(expr = -sub.w[3]-sub.w[4]>= -crop_harvest_coefficients[s][2]*value(master.x[3]))
    sub.c4 = Constraint(expr = -sub.w[3] >= -farmer_quota)

    sub.dual = Suffix(direction=Suffix.IMPORT)
    solver = SolverFactory('cplex')
```

برای ایجاد برش بهینگی لازم است مقادیر دوگان را داشته باشیم که کد زیر آن را تولید می کند همچنین برای چک کردن کد مقادیر بهینه زیر مسیله را به ازای ایکس نمایش می‌دهد.

```

#display result
print(value(master.x[1]))
for i in sub.sale_index:
    print(f"{sub.w[i]}:{value(sub.w[i])}")
for i in sub.purchase_index:
    print(f"{sub.y[i]}:{value(sub.y[i])}")

#dual
dual_var_2=[]
for c in sub.component_objects( Constraint, active = True ):
    cobject = getattr( sub, str( c ) )
    for index in cobject:
        dual_var_2.append(sub.dual[cobject[index]])

simplex_multiplier[s] = dual_var_2
print(simplex_multiplier)

```

در ادامه مقادیر لازم برای ساخت برش محاسبه می‌شوند و اگر به شرط بهیمنگی نرسیده باشد کات اضافه می‌شود در غیر اینصورت به شمارنده تعداد سناریوهای که به بهیمنگی به ازای این ایکس رسیده اند اضافه می‌شود و حلقه فور به پایان می‌رسد.

$$E_{nooc+1} = (\delta_s^v) t_s p_s$$

$$e_{nofc+1} = (\delta_s^v) h_s p_s$$

$$E_{nooc+1} x + \theta_s \geq e_{nofc+1}$$

در مورد شرایط پایان این مقدار چک میکند که با تولید نقطه گوشه جدید در سناریو ایا این نقطه مجدد تولید شده است. به دلیل اینکه در گام های نهایی اختلاف ها خیلی کم است و برای مقدار کم تفاوت ممکن است تعداد گام ها زیاد شود در نقطه ای که اختلاف این مقادیر زیر $0,00001$ باشد پذیرش کرده ایم.

```

# e, E calculate
e = (scenario_prob[s] * np.array([simplex_multiplier[s]] @ np.array(b[s]))
E = (scenario_prob[s] * np.array([simplex_multiplier[s]] @ np.array(t[s]))
w_this_iteration = e - E @ np.array([value(master.x[1]), value(master.x[2]), value(master.x[3])])
w[s].append(w_this_iteration)
print(f"    > w = {w_this_iteration}, theta = {value(master.theta[s])} ")
print(f"    > e = {e}")
print(f"    > E = {E}")

# feasibility cut
if w_this_iteration > value(master.theta[s]) + 0.00001:
    nooc += 1
    master.optimality_cut.add(list(E @ np.array([master.x[1], master.x[2], master.x[3]])) @ [0] + master.theta[s] >= e[0])
    print("\n✅ Added optimality cut to master problem.")
    master.optimality_cut.pprint()
    print("-" * 50 + "\n")
else:
    optimal_scenario_counter += 1

```

بررسی بهینگی کلیه سناریوها

پس از خروج از حلقه چک می‌شود که آیا در این تکرار تمام زیر مسائل بهینه شده اند یا خیر و اگر شرط برقرار شود به جواب بهینه رسیده ایم و از حلقه وایل خارج می‌شویم. در غیر اینصورت آتقدر این روند تکرار می‌شود تا بالاخره به جواب بهینه برسیم.

```

optimal_scenario_counter += 1
if optimal_scenario_counter == scenario:
    print("\n🔴 Termination Criteria Met!")
    print(f"✅ Final Master Problem Objective: {value(master.obj):.2f}")
    print(f"✅ Final Decision Variables: x1 = {value(master.x[1]):.2f}, x2 = {value(master.x[2]):.2f}, x3 = {value(master.x[3]):.2f}")
    print("-" * 50 + "\n")
    not_stop_condition = False

```

اگر در ۵۰ ایتريشن نرسیم الگوریتم متوقف می‌شود و فرض می‌شود اعداد به گونه ای است که مستر پرابلم ناموجه نمی‌شود. همچنین با مدلسازی درست امکان ندارد زیرمسائل ناموجه شوند.

برای اطمینان از اینکه مستر پرابلم ناموجه نمی‌شود می‌شود شرط گذاشت که اگر ناموجه شود خطا دهد و از برنامه خارج شود.

نتیجه اجرا بر داده های ۵۰ تایی

با اجرای این تابع داده هایی تولید شده اند. این داده ها و خروجی کد در فایل وردی پیوست می‌شود و برای جلوگیری از شلوغی اینجا آورده نمی‌شوند.

با اجرای کد مقادیر t و h ساخته میشود مثلاً برای سناریو اول

```

t:{1: array([[0.4005, 0.    , 0.    ],
            [0.    , 0.4806, 0.    ],
            [0.    , 0.    , 3.204 ],
            [0.    , 0.    , 0.    ]]).....

```

و $h: \{1: [200, 240, 0, -6000]\dots$

پس از اجرا کد پس از ۷ تکرار به نتیجه زیر می‌رسیم

● Termination Criteria Met!

✓ Final Master Problem Objective: -528,449.18

✓ Final Decision Variables: $x_1 = 123.04$, $x_2 = 109.10$, $x_3 = 267.86$

برای نمایش نحوه کار مثلاً برای تکرار ده چک کردن بهینگی دو تکرار اول خروجی زیر را داریم و در نهایت پس از چک شدن بهینگی ۶۴ سناریو خروجی اضافه کردن کد بهینگی برای دو سناریو اول به صورت زیر است.

```
=====
[Icon] Iteration 2
-----
Master Problem Objective Value : -647,738.91
Decision Variables:
x[1]: 0.0
x[2]: 0.0
x[3]: 500.0
=====
-----feasibility check-----

-----
| scenario1
-----

♦ Scenario 1 feasibility check:
|  $\pi$  (Dual values) = [0.0, 0.0, -0.0, -0.0]
current x is feasible.

-----
| scenario2
-----

♦ Scenario 2 feasibility check:
|  $\pi$  (Dual values) = [0.0, 0.0, -0.0, -0.0]
current x is feasible.
```

و برای بهینگی این دو سناریو بهینه شده‌اند

•

```
-----sub problem solving for feasible x = (0.0, 0.0, 500.0)-----  
  
-----  
scenario1  
-----  
0.0  
w[1]:0.0  
w[2]:0.0  
w[3]:1602.0000000000002  
w[4]:0.0  
y[1]:200.0  
y[2]:240.0  
{1: [237.64775999999998, 209.6892, 35.94672, -0.0]}  
  ➤ w = [4.02683146],theta =4.026831455999998  
  ➤ e = [9.785496]  
  ➤ E = [[0.00951779 0.01007766 0.01151733]]  
  
-----  
scenario2  
-----  
0.0  
w[1]:0.0  
w[2]:0.0  
w[3]:1449.9999999999993  
w[4]:0.0  
y[1]:200.0  
y[2]:240.0  
{2: [243.90264904319997, 215.208219744, 36.8928376704, -0.0]}  
  ➤ w = [28.16153276],theta =28.161532755072002  
  ➤ e = [60.25830153]  
  ➤ E = [[0.05304883 0.05616935 0.06419354]]  
  
-----
```

اما مثلاً از سناریو ۱۷ برش لازم است

```

scenario17
0.0
w[1]:0.0
w[2]:0.0
w[3]:6000.0
w[4]:459.0
y[1]:200.0
y[2]:240.0
{17} [667.5127975715615, 588.9818802182011, 28.046756200485788, 72.02156612126298]]
  ▶ w = [-481.35525797], theta = -528.2346563550807
  ▶ e = [-227.73966835]
  ▶ t = [[1.58901261 1.50777920 0.5072312 ]]
```

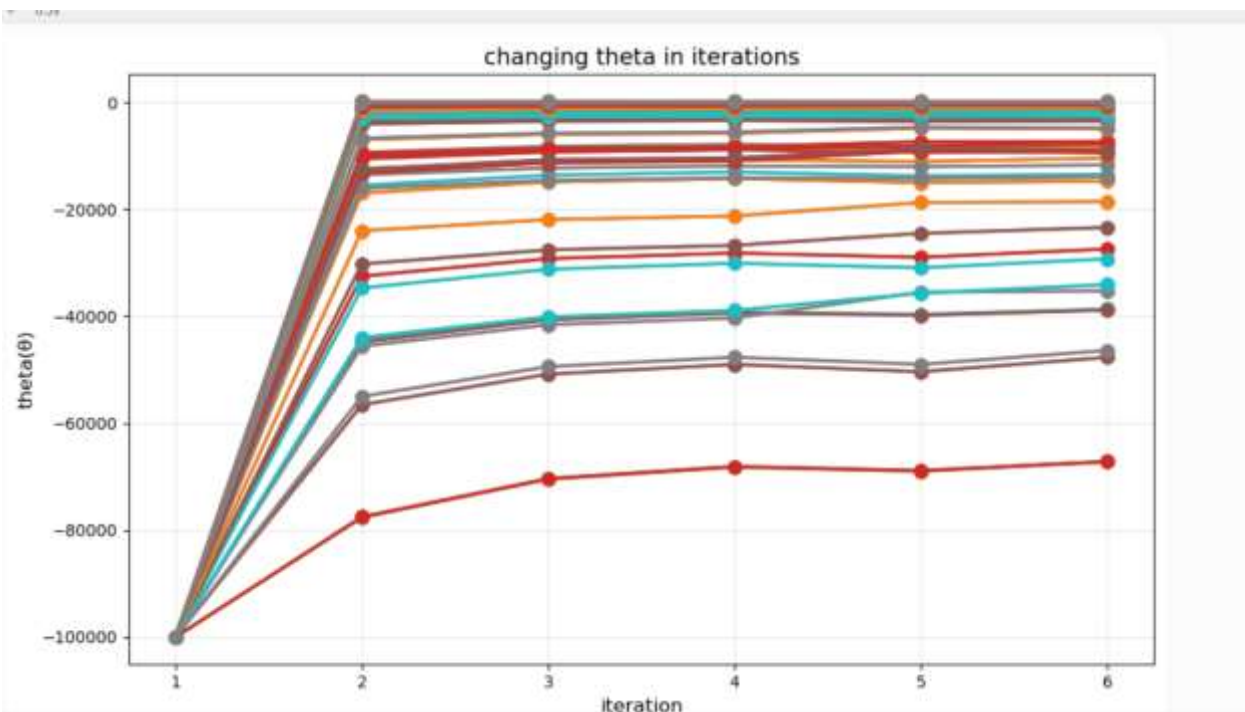
Added optimality cut to master problem.

optimality_cut : size=65, index=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36,

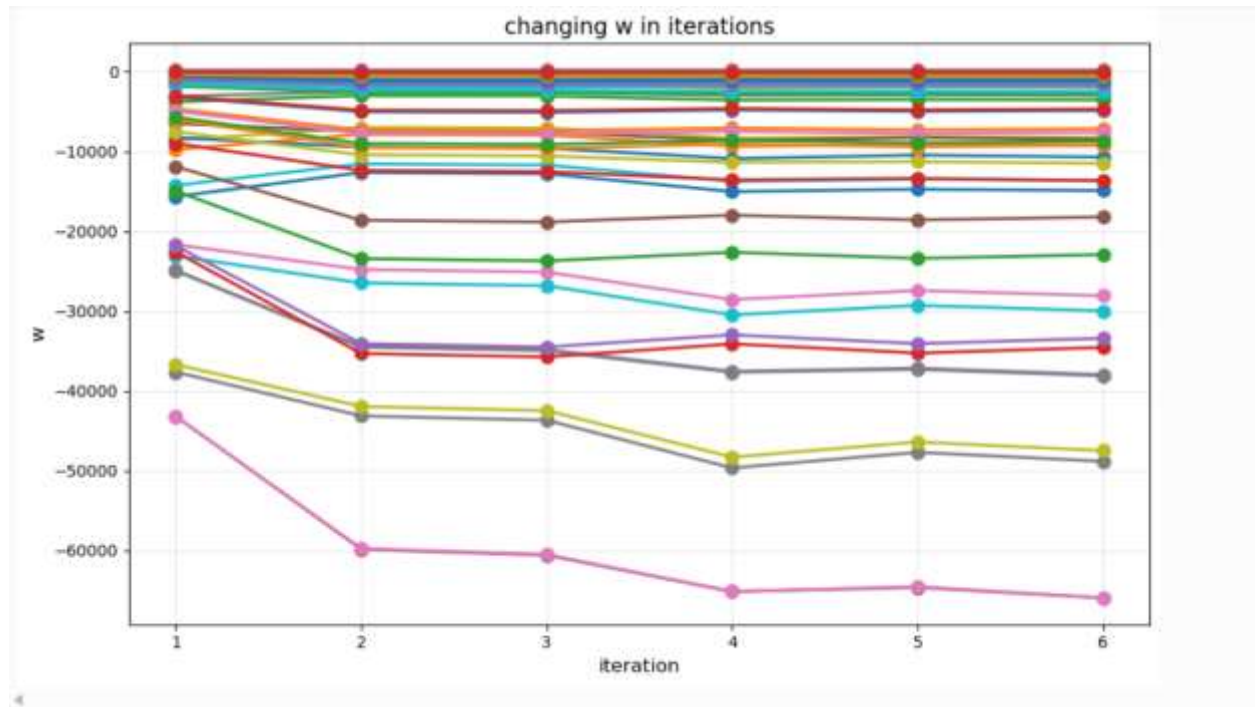
Key	lower	Body	Upper	Active
1	0.785690	$0.809517702780000001 \times x[1] + 0.018077662951099909 \times x[2] + 0.011517329088000003 \times x[3] + \theta[1]$	$+\infty$	True
2	68.25830152831999	$0.05104882618589596 \times x[1] + 0.056169345351184 \times x[2] + 0.06419351754848597 \times x[3] + \theta[2]$	$+\infty$	True
3	179.09259323679183	$0.16560308326763467 \times x[1] + 0.1753444411869076 \times x[2] + 0.20030364697932285 \times x[3] + \theta[3]$	$+\infty$	True
4	316.22021002633863	$0.360081685736006 \times x[1] + 0.31273354908289335 \times x[2] + 0.363124056689821 \times x[3] + \theta[4]$	$+\infty$	True
5	271.85361107332903	$0.2688729304900609 \times x[1] + 0.28468880513900553 \times x[2] + 0.3254588401588636 \times x[3] + \theta[5]$	$+\infty$	True
6	193.68136379025395	$0.17427172426184484 \times x[1] + 0.38452308225960040 \times x[2] + 0.2108834318395433 \times x[3] + \theta[6]$	$+\infty$	True
7	94.92932352237971	$0.09025744467169525 \times x[1] + 0.89556670612297144 \times x[2] + 0.10921969271196732 \times x[3] + \theta[7]$	$+\infty$	True
8	10.392872863500727	$0.015077440458911873 \times x[1] + 0.021046701662377272 \times x[2] + 0.024053373528431164 \times x[3] + \theta[8]$	$+\infty$	True
9	107.84771308758833	$0.2595580330200133 \times x[1] + 0.27482678790439047 \times x[2] + 0.3148877570858175 \times x[3] + \theta[9]$	$+\infty$	True
10	431.0088088319322	$1.0723181950379732 \times x[1] + 1.1351957359225595 \times x[2] + 1.2975951267086392 \times x[3] + \theta[10]$	$+\infty$	True
11	1636.2847834552335	$0.044370036020041 \times x[1] + 4.282274155786878 \times x[2] + 4.894827606613574 \times x[3] + \theta[11]$	$+\infty$	True
12	2945.3159195594317	$7.30669765718133 \times x[1] + 7.7365634818484655 \times x[2] + 8.8417181735411 \times x[3] + \theta[12]$	$+\infty$	True
13	2621.2787228550355	$6.3245645514497895 \times x[1] + 6.696597760358598 \times x[2] + 7.653254581266969 \times x[3] + \theta[13]$	$+\infty$	True
14	1945.9872146099954	$4.685798213694111 \times x[1] + 4.96143340273494 \times x[2] + 5.670209603125645 \times x[3] + \theta[14]$	$+\infty$	True
15	683.408082012721	$1.695389006884558 \times x[1] + 1.7951177719954141 \times x[2] + 2.051563679947584 \times x[3] + \theta[15]$	$+\infty$	True
16	201.94253210067094	$0.5066161153754879 \times x[1] + 0.5364170633387517 \times x[2] + 0.6130400723875449 \times x[3] + \theta[16]$	$+\infty$	True
17	384.80149507066477	$1.5090128058301502 \times x[1] + 1.5977782649966292 \times x[2] + 1.8260323028532908 \times x[3] + \theta[17]$	$+\infty$	True
18	2254.1825501837575	$8.876880860584318 \times x[1] + 0.398975156030474 \times x[2] + 18.741685892606256 \times x[3] + \theta[18]$	$+\infty$	True

نمودار

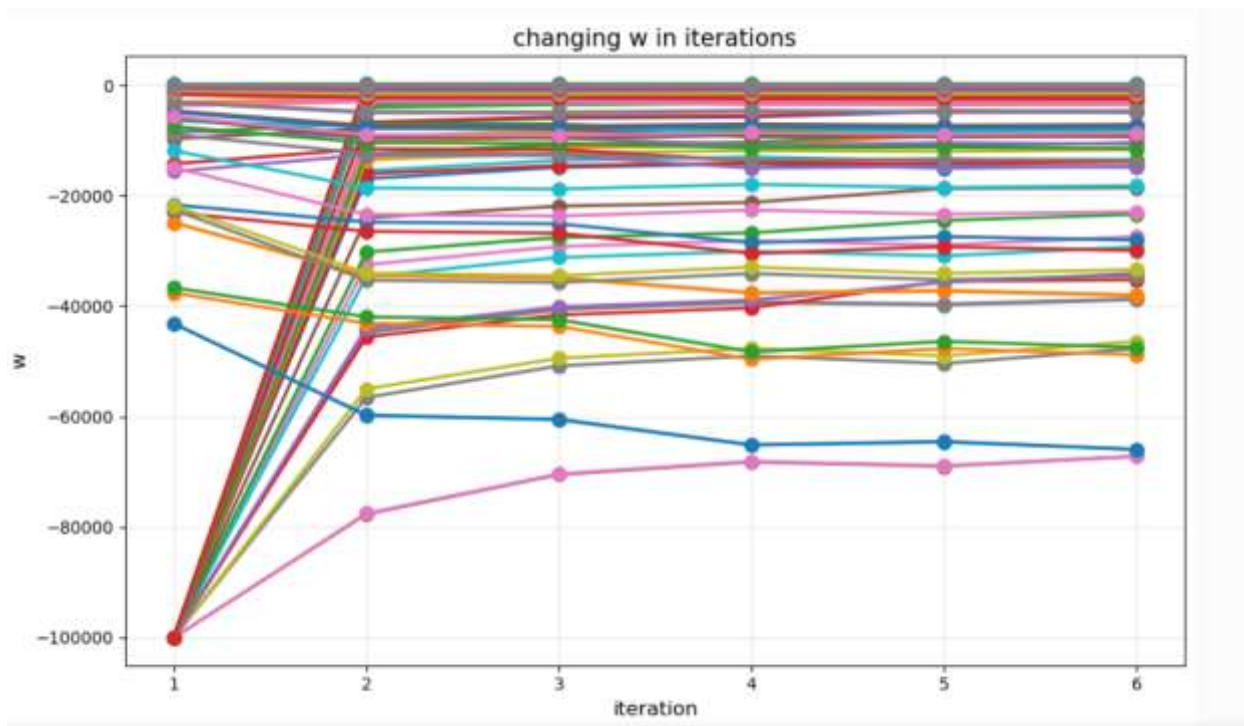
نمودار زیر تغییرات مقدار تتا در تکرارهای مختلف را برای ۶۴ سناریو نمایش می‌دهد. در بسیاری از سناریو ها مقدار تتا خیلی سریع در گام اول همگرا شده. پس از آن در تکرارهای بعدی تغییرات تتا به مرور کم می‌شود. این نشان می‌دهد در گام های نهایی معمولاً الگوریتم کند است که ضعف مسائل بندرز کلاسیک می‌باشد



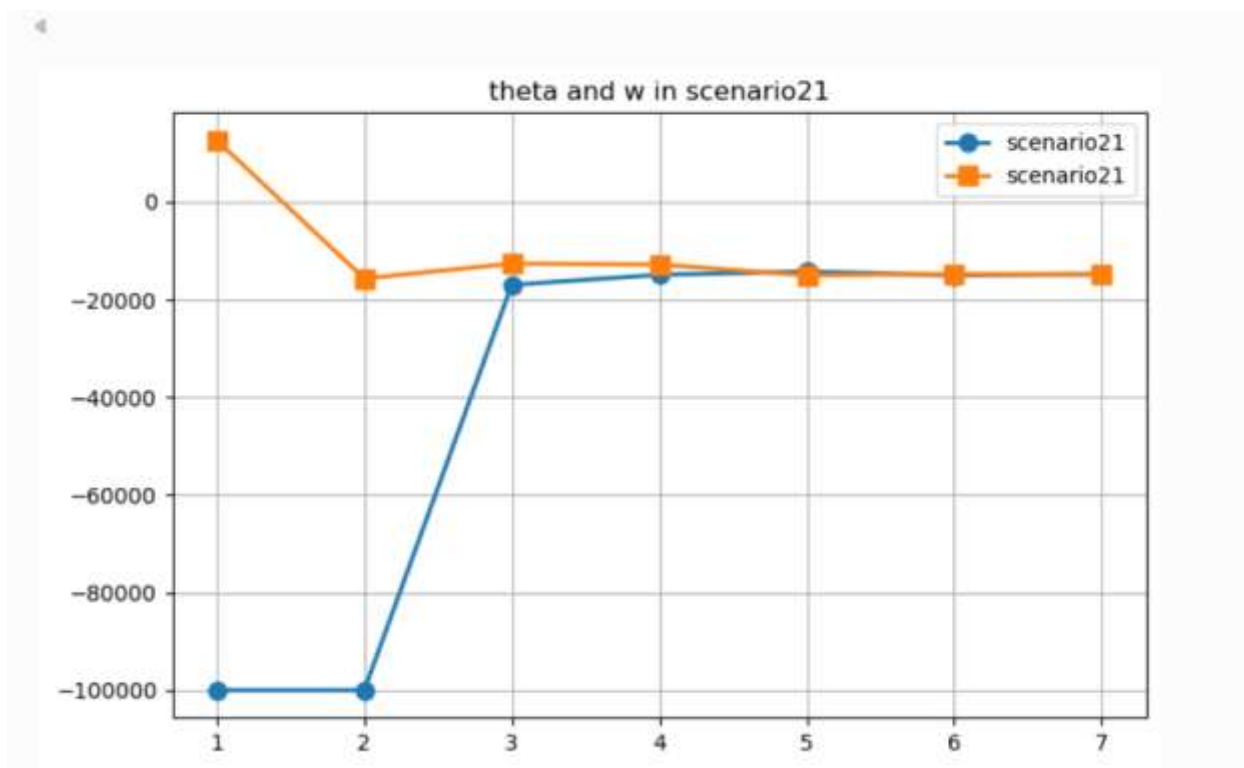
نمودار زیر تغییرات مقدار دبلو در تکرارهای مختلف را برای ۶۴ سناریو نمایش می‌دهد. که مشابه تتا دارای روند همگرایی سریع در آغاز و کند در انتها است



نمودار زیر این دو نمودار را روی هم می‌اندازد.



چون نمودار شلوغی است برای تکرار خاصی کشیده می‌شود. دبلو و تتا به گونه ای کران بالا و پایین هر سناریو را نمایندگی می‌کنند. و به همگرا می‌شوند. تتا نمایانگر ایجاد یک نقطه گوشه جدید است و دبلو یک نقطه موجه در تابع مستر پرابلم است.



نتیجه‌گیری

روش برای ۶۴ سناریو با عدم قطعیت در قیمت خرید و فروش (تابع هدف) و ضریب بهره‌وری محصولات (ضرایب متغیر محدودیت اول) با سه بار اجرا با میانگین ۱۱۱ ثانیه [۹۷،۹۴،۱۴۰] ثانیه حل شد که عدد بسیار مطلوبی است.